

Bug report — HW05 Performance Testing

- **Sinh viên:** Lê Nhựt Duy — **MSSV:** 23127178
- **SUT:** EShop backend API :3000 — <https://github.com/ttbhanh/eshop-sut>
- **GitHub Issues:** https://github.com/DuyITLOR/group05_eshop/issues

§6: "Log any genuine bugs or performance issues (error responses, crashes, functional regressions) on your GitHub Issues page with screenshots. Logging performance issues such as high latency or elevated error rate is encouraged but not penalised if absent."

Tóm tắt: 2 bug chức năng đã xác nhận bằng request thật và **đã mở Issue #288, #289** · 1 ứng viên đã bị loại sau khi kiểm · 3 defect cũ ảnh hưởng tới cách đọc số liệu · 0 lỗi hiệu năng (4 lượt chạy đều 0% error).

1. Bug đã xác nhận

BUG-P1 — GET /api/orders/:id không kiểm xác thực (IDOR)

Endpoint	GET /api/orders/:id
Đặc tả nơi	api_specification.md §4 xếp endpoint này dưới dòng "Yêu cầu Header: Authorization: Bearer <token> "
Thực tế	Không có middleware authenticateToken → không cần token, và đọc được đơn hàng của bất kỳ người dùng nào chỉ bằng cách đổi :id
Nguyên nhân	server.js:344 — thiếu authenticateToken, trong khi mọi route order khác đều có
Loại	bảo mật (IDOR) + lệch đặc tả
Ảnh	screenshots/bug-evidence-verify-bugs.png — ảnh chụp toàn bộ output verify-bugs.sh, gồm cả ba khối
GitHub Issue	#288 — có ảnh nhúng sẵn

Bằng chứng (chạy lại được bằng bash bug-report/verify-bugs.sh):

```
-- gọi KHÔNG kèm Authorization header:
HTTP 200
{"id":1,"user_id":3,"total_amount":500000,"status":"pending",
  "shipping_address":"1 Đường Perf, Q1, TP.HCM","created_at":"2026-08-13 05:08:32"}

-- doi chieu trong DB: order 1 thuoc ai?
1|3|perf_23127178_001@eshop.test|500000      ← đơn của user khác, đọc được không cần token

-- doc them don cua nguoi khac, van khong token:
order 5 → HTTP 200  user_id=7  total=540000
```

```
order 12 → HTTP 200 user_id=14 total=610000
order 33 → HTTP 200 user_id=5 total=520000

-- doi chung: endpoint order khác C0 kiem token:
GET /api/orders/my-orders khong token → HTTP 401
```

Dòng đối chứng cuối là phần quan trọng nhất: nó chứng minh đây **không phải** thiết kế "API này vốn công khai" mà là **một route bị bỏ sót**, vì route order ngay bên cạnh vẫn chặn đúng.

BUG-P2 — `POST /api/coupon-usage` tồn tại và ghi DB nhưng không có trong tài liệu API

Endpoint	<code>POST /api/coupon-usage</code>
Đặc tả nói	Không đề cập. <code>api_specification.md</code> §5 chỉ có <code>POST /api/apply-coupon</code> (dòng 154) và <code>GET /api/coupons</code> (dòng 166)
Thực tế	Route tồn tại, cần token, ghi thật vào bảng <code>coupon_usage</code> (server.js:444)
Loại	tài liệu thiếu — ảnh hưởng trực tiếp tới việc chọn phạm vi kiểm thử: không ai test được endpoint mình không biết là có
Ảnh	screenshots/bug-evidence-verify-bugs.png — cùng ảnh, khối thứ hai
GitHub Issue	#289 — có ảnh nhúng sẵn

Bằng chứng:

```
-- tìm trong api_specification.md:
0 lần → KHÔNG được tài liệu hoá

-- tìm trong server.js:
444:app.post("/api/coupon-usage", authenticateToken, (req, res) => {
-- gọi thu thập:
response      : {"message":"Usage recorded"}
coupon_usage  : 4 → 5 dòng      ← endpoint có ghi DB thật
```

Đáng lưu ý thêm: endpoint này ghi `coupon_usage` mà **không kiểm** coupon có tồn tại hay người dùng đã dùng quá `max_uses_per_user` chưa — trong khi `POST /api/apply-coupon` thì có kiểm ([server.js:391](#)). Tức là ghi thẳng vào bảng này bỏ qua được toàn bộ luật giới hạn số lần dùng coupon. Chưa đưa vào bug riêng vì cần đọc kỹ hơn phần nghiệp vụ coupon, nhưng đã ghi lại ở đây.

2. Ứng viên đã LOẠI sau khi kiểm

Mục này quan trọng không kém mục 1: báo một hành vi đúng thành bug làm mất tin cậy của cả bug report.

ĐÃ LOẠI — "`import-products` báo số dòng đã insert nhỏ hơn thực tế"

Nhận định ban đầu (từ ĐỌC CODE): `stmt.finalize(() => res.json(...))` ([server.js:234](#)) trả response trước khi các callback của `stmt.run(...)` chạy xong, nên biến `inserted` vẫn còn 0 lúc

serialize JSON.

Kiểm bằng request thật → nhận định đó SAI:

Gửi đi	Server báo	DB thực tế	Khớp?
5 sản phẩm	Import hoàn tất: 5/5 , inserted: 5	+5 dòng	Đúng
60 sản phẩm	Import hoàn tất: 60/60 , inserted: 60	+60 dòng	Đúng
3 sản phẩm, 1 dòng thiếu name	Import hoàn tất: 2/3 , inserted: 2 , errors: ["Hàng 3: Thiếu tên sản phẩm"]	+2 dòng	Đúng

Vì sao đọc code lại sai: node-sqlite3 xếp mọi lệnh trên cùng một database handle theo thứ tự, và callback của finalize chạy sau các lệnh đã xếp trước nó. Không có race như suy luận ban đầu.

Hệ quả cho test plan: vẫn không assert theo inserted , nhưng vì một lý do khác hẳn — đó là con số phụ thuộc dữ liệu (dòng CSV thiếu name bị bỏ qua một cách hợp lệ), nên assert theo nó là biến một đặc điểm dữ liệu thành "lỗi hiệu năng". Assert theo HTTP status + field message .

Bài học: nhận định này đã lan vào 5 file tài liệu trước khi bị bắt, vì nó nghe hợp lý và có kèm số dòng code. Đọc code cho ra giả thuyết, không cho ra kết luận. Ghi thành lỗi #9 trong ai-audit/ai-audit-repo rt.md .

ĐÃ LOẠI — các quan sát khác

Ứng viên	Vì sao KHÔNG phải bug
PUT /api/admin/orders/:id/status trả 400 cho phần lớn request trong lượt dài	Đúng đặc tả FR-10: một order chỉ chuyển tiếp một lần cho mỗi trạng thái (server.js:537-551). Con số "18,25% error" ở một lượt chạy trước là lỗi của test plan của tôi, không phải của SUT
Hàng loạt 403 ở lượt chạy đầu	Account lockout hoạt động đúng như code. Nguyên nhân thật: users.csv của tôi chứa 2 dòng mật khẩu sai và luồng chính đọc chính file đó → tự khóa tài khoản của mình
GET /api/admin/orders chậm hơn GET /api/admin/users (p95 17ms vs 13ms ở Stress)	Đúng như thiết kế: JOIN orders × users , không phân trang (server.js: 510). Chậm hơn không có nghĩa là sai — và 17ms còn cách rất xa mọi ngưỡng đã đặt

3. Defect đã biết từ HW trước — ảnh hưởng tới cách đọc số liệu

Không mở Issue mới, chỉ ghi vì chúng thay đổi cách đọc kết quả đo.

Defect	Vị trí	Ảnh hưởng tới bài đo
login_attempts cộng 2 thay vì 1 → lockout sau 2 lần sai, không phải 3	serve r.js:54	Mọi 403 trong lượt chạy gần như luôn là lockout — hành vi chức năng. Phải tách khỏi error rate

Defect	Vị trí	Ảnh hưởng tới bài đo
Mật khẩu lưu plaintext, so sánh bằng ===	<code>serve</code> <code>r.js:4</code> <code>6</code>	Đây là giới hạn quan trọng nhất của toàn bộ bài đo. Không có bcrypt/argon2 nên login không tốn CPU lắm. p95 24ms của <code>POST /api/login</code> ở mức 200 VU không đại diện cho một hệ thống băm mật khẩu đúng cách — ở đó login thường là endpoint đắt nhất, không phải rẻ nhất
Không có rate limiting ở bất kỳ route nào	toàn bộ server .js	Mọi 4xx đến từ credential/token/body, không phải throttling

4. GitHub Issues — đã mở

Issue	Bug	Label
#288	BUG-P1 — IDOR ở <code>GET /api/orders/:id</code>	<code>security</code> , <code>performance-testing</code>
#289	BUG-P2 — <code>coupon-usage</code> không có trong tài liệu	<code>documentation</code> , <code>performance-testing</code>

Cả hai issue **có ảnh hưởng sẵn**, không phải kéo-thả tay. Ảnh nằm trên nhánh `evidence/23127178-hw05` của repo SUT, nhúng bằng raw URL — cùng cách HW03 và HW04 đã dùng.

Tạo lại được bằng:

```
DRY_RUN=1 bash bug-report/create-github-issues.sh # xem trước
bash bug-report/create-github-issues.sh          # tạo thật (chỉ chạy một lần)
```

Nội dung issue nằm ở `issue-p1.md` và `issue-p2.md` — sửa được như tài liệu bình thường, script chỉ chèn ảnh vào chỗ `__IMAGE__` rồi gửi lên.

5. Vấn đề hiệu năng: không có

Cả 4 lượt chạy đều **0% error**, kể cả Stress ở 200 VU / 550 req/s. p95 cao nhất đo được là **26ms** (Stress, toàn workflow) và **40ms** (`POST /api/login` ở bậc 200 VU). Không có mẫu nào trả 500, không có timeout, không có connection refused.

Điều này **không** có nghĩa "SUT chịu tải tốt vô hạn" — nó có nghĩa **lượt Stress chưa chạm được giới hạn của SUT**, vì giới hạn gặp trước là của load generator: JMeter tiêu CPU đỉnh 60,9% trong khi `node` chỉ 19,7%. Phân tích đầy đủ ở `report/main-report.md §3`.

Theo §6 thì mục này *khuyến khích, không bắt buộc* — và báo cáo "không tìm thấy vấn đề hiệu năng, kèm lý do vì sao phép đo chưa đủ chạm giới hạn" trung thực hơn là nặn ra một con số để có cái mà ghi.